

基于FPGA与RK3588图像分类处理系统设计与实现

袁小平, 涂齐阳

(中国矿业大学 信息与控制工程学院, 江苏 徐州 221116)

摘要:为解决 ViT 模型在基于 CPU 侧平台运行效率低、延迟长的问题, 该文设计了一款基于 FPGA (field-programmable gate array, 现场可编程门阵列)+NPU (neural network processing unit, 神经网络处理器) 的图像采集与处理系统 (简称“FPGA 系统”)。系统首先利用 FPGA 时序控制与计算的稳定性, 适配图像传感器接口像素传输带宽, 将 patch-embedding 层分解为向量式计算处理, 并利用流水线设计对模型叠加项融合后的张量进行累计输出, 得到 patch-embedding 的计算结果后, 将输出叠加位置标志拼接为位置标识报文; 随后利用 UDP (user datagram protocol, 用户数据报协议) 的协议栈从 FPGA 输出 patch-embedding 神经网络结构的计算结果, 同时实现并发的图像采集、视频呈现、UDP 传输任务; 最后, 基于 RK3588 的 NPU 实现了多线程和线程池设计、UDP 接收、线程池处理注意力计算任务。在划分设计流程、验证设计、优化调试后, 每帧运行平均时间由最初 0.643 9 s 提升至 0.099 5 s, FPGA 系统 Block RAM 资源用量为 56.42%, 且未出现时序违例, 对行人识别精度的平均值为 0.85, 对车辆识别平均精度为 0.905。

关键词: FPGA; RK3588; 向量式计算; 叠加项融合; 位置标识报文; 多线程设计

中图分类号: TP391.4 **文献标识码:** A **文章编号:** 1002-4956(2025)09-0091-12

随着计算机视觉技术的快速发展, 基于 Transformer 的视觉模型 (如 ViT, Vision Transformer 等) 逐渐取代了卷积神经网络 (convolutional neural networks, CNN) [1-2]。ViT 模型可以通过自注意力机制捕捉全局上下文信息, 在图像分类、目标检测等任务中表现出了显著优势[3-4], 然而该模型存在计算复杂度高、参数量大等问题, 在实时性要求高的嵌入式场景 (如自动驾驶、机器人视觉) 中面临挑战[5-6]。现场可编程门阵列 (field-programmable gate array, FPGA) 凭借高并行性、低功耗和可重构等优势而成为部署加速深度学习模型的重要硬件载体[7-8]。例如, 基于 FPGA 的实时图像预处理算法已在机器人视觉系统中实现了高效边缘检测、畸变矫正等功能[9-10]。

CNN[11]也广泛应用了 FPGA 加速设计[9,12]。然而, 该模型参数量大、内部的数字信号处理单元 (digital signal processor, DSP) 和随机存取存储器 (random access memory, RAM) 资源紧张[13-14], 需要用片外存储资源或异构系统处理计算任务[15], 而异构系统处理器之间的通信需要额外开销, 会影响计算任务的实时

性[16]。因此, 探索 ViT 模型在 FPGA 与嵌入式系统中的优化与实时性应用, 成为了解决计算资源受限场景下模型部署难题的关键[17-18]。

1 系统架构与设计流程

1.1 系统架构

图 1 为基于 FPGA+NPU (neural network processing unit, 神经网络处理器) 的图像采集与处理系统 (简称“FPGA 系统”) 的架构。为提高设计效率, 本工作将 Embedding 层、class_token 拼接、位置嵌入操作部署于 FPGA, 并利用 RK3588 嵌入式 NPU 实现了算法深层网络的运算, 利用 UDP (user datagram protocol, 用户数据报协议) [19]协议栈将运算数据传送给嵌入式 NPU 端进行处理, 并采用多线程设计与队列缓存, 实现数据实时接收功能, 以及解决队列快写慢读的存算累计延迟问题, 在嵌入式端实现剩余网络层的运算任务。为方便观察图像采集和识别情况, 通过 HDMI (high definition multimedia interface, 高清多媒体接口) 输出图像采集信息, 而嵌入式端则通过通用异

收稿日期: 2025-03-04 修改日期: 2025-04-08

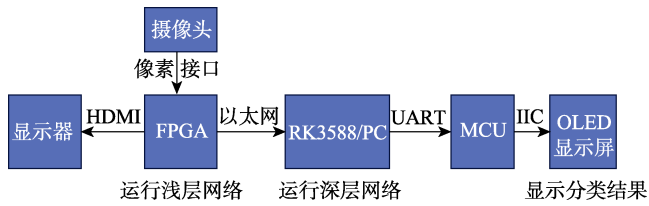
基金项目: 中国矿业大学 2023 年国家级大学生创新训练项目 (202310290032Z); 中国矿业大学智慧课程建设 (2024KC19)

作者简介: 袁小平 (1966—), 男, 江苏仪征, 博士, 教授, 主要研究方向为信号处理、智能算法硬件加速等, xpyuankd@163.com。

引文格式: 袁小平, 涂齐阳. 基于 FPGA 与 RK3588 图像分类处理系统设计与实现[J]. 实验技术与管理, 2025, 42(9): 91-102.

Cite this article: YUAN X P, TU Q Y. Image classification processing system based on FPGA and RK3588[J]. Experimental Technology and Management, 2025, 42(9): 91-102. (in Chinese)

步接收发送设备 (universal asynchronous receiver/transmitter, UART) 将运算数据传送给微控制单元 (micro control unit, MCU), 用于执行其他任务和显示分类结果。



注: IIC 为 inter-integrated circuit (集成电路系统)。

图 1 FPGA 系统的架构

FPGA 系统各子系统的性能需求如表 1 所示。

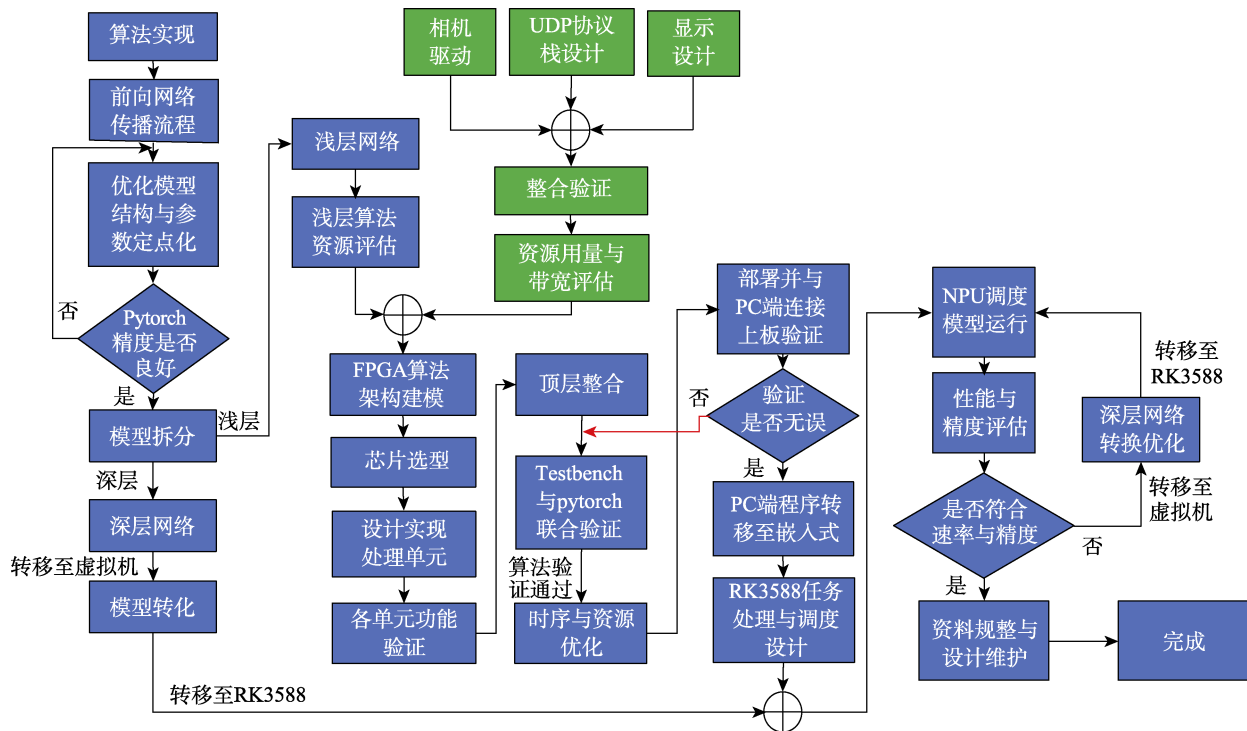
1.2 设计流程

FPGA 系统的设计流程见图 2。系统需要同步驱动相机、采集图像和呈现视频, 因此, 设计系统时需要设计 UDP 报文与算法, 在训练与推理算法后, 软件算法团队需要与 FPGA 团队共同交流, 明确前向传播的计算流程, 针对 FPGA 系统的计算特性, 对算法进

表 1 各子系统功能和性能需求

子系统名称	子系统功能任务	性能要求
FPGA 系统	采集图像、显示图像、计算预处理与 ViT 浅层网络、传输计算结果	相机在 60 Hz 帧率下, 完成 patch-embedding 的计算与传输任务
RK3588/PC	接受计算结果、计算 ViT 深层网络、计算结果串口输出	不丢帧接收数据, 模型计算帧率为 60 Hz
MCU	接受和显示分类结果、执行其他控制任务	无

行定点转化调优, 并优化部分不利于 FPGA 系统的算法结构。随后, 根据 FPGA 和 RK3588/NPU 的负载程度, 将模型拆分为浅层网络和深层网络。对需要部署于 FPGA 的浅层网络进行算子层面的动作剖析与资源评估, 在完成了采集与传输显示任务后, 根据传输和采集接口时序和带宽, 对 FPGA 模型处理架构进行建模, 根据算法层模型及接口系统的资源用量来对芯片进行选型。之后根据芯片资源总量和分布、电气特性, 设计实现算法的时钟、数据、控制、复位架构, 完成后再验证各个单元的功能, 验证通过后整合顶层设计。



注: ⊕ 表示等待任务都完成。

图 2 FPGA 系统的设计流程

要判断系统架构的处理结果是否正确, 需要模拟相机图像接口的时序及复位动作, 设计仿真文本, 在 pytorch 框架中将图像改为 1 024 px×768 px, 将处理后的图像数据保存为文本文件, 再于 FPGA 开发软件中读取此.txt 文件, 将之解析为二进制数据后存储。按

照 ov5640 图像传感器的接口时序逻辑从文本中读取像素信息, 传递给处理单元, 将处理单元输出得到的结果存入另一个文本文件, 在 pytorch 框架中读取该结果, 并将该结果转化为十进制数据, 以此十进制数据作为深层网络的输入数据, 对比基于 FPGA 处理单

元和 pytorch 框架处理后的图像分类精度, 两者精度相差的绝对值越小, 表明整体处理架构设计越正确、与理论相比的计算误差越小。

在功能验证无误后, 对 FPGA 系统进行综合 (将 Verilog 描述的设计转换为门级网表) 和布局布线 (根据设计的逻辑网表, 将逻辑元件如逻辑门、存储单元等映射到 FPGA 芯片上的物理位置) 后, 再对布局布线结果的时序违例 (不满足触发器的建立和保持时间) 情况进行代码级资源优化和时序优化, 优化完成后上板验证。在 PC 端编写千兆以太网读取程序, 用千兆以太网将 FPGA 的计算结果输入 PC, 验证 ViT 算法中的 Transformer Encoder 神经网络结构计算过程进行验证, 若有问题则检验验证程序与 PC 程序的正确性, 验证无误则进行 RK3588 的模型转换与任务调度设计, 用 NPU 部署深层网络并评估性能。优化 NPU 后, 完

成系统设计和开发任务。

2 ViT 模型结构程序介绍与参数定点化

2.1 模型网络结构

ViT 模型网络结构如图 3 所示^[1], 整体前向传播网络包括 Embedding 层、Transformer Encoder 层、MLP Head 层。模型的输入数据维度为 $[1, 3, 224, 224]$ 。这种结构依托注意力机制, 利用全局特征对图像实现分类, 提高了图像分类的准确性。

2.2 前向传播程序介绍

图 4 展示了 ViT 模型前向传播程序的框架, 输入张量经过卷积后得到 $[1, 14, 14, 768]$ 维度的张量, 经过降维操作、class_token 拼接、位置嵌入后得到数据维度 $[1, 197, 768]$, 将数据输入 Transformer Encoder 层, 再经过归一化操作 (Norma_layer)、全连接结构 (Linear) 后可得到输出分类。

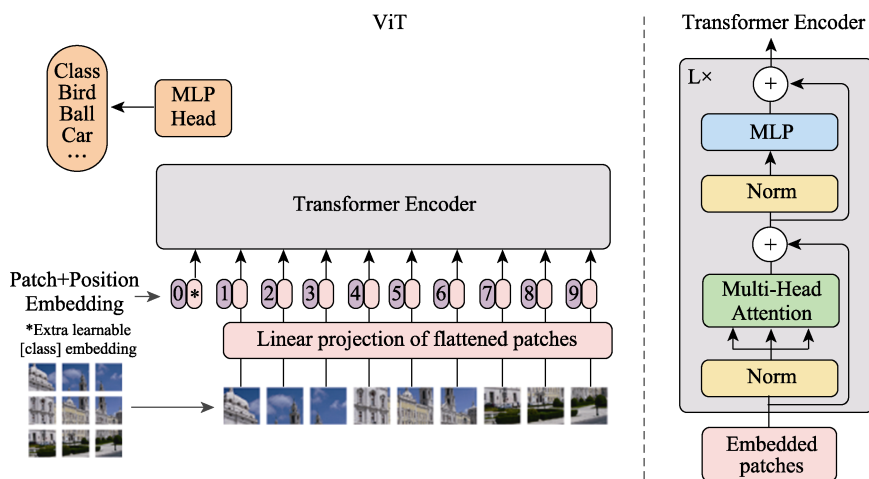


图 3 算法原理的网络结构^[1]

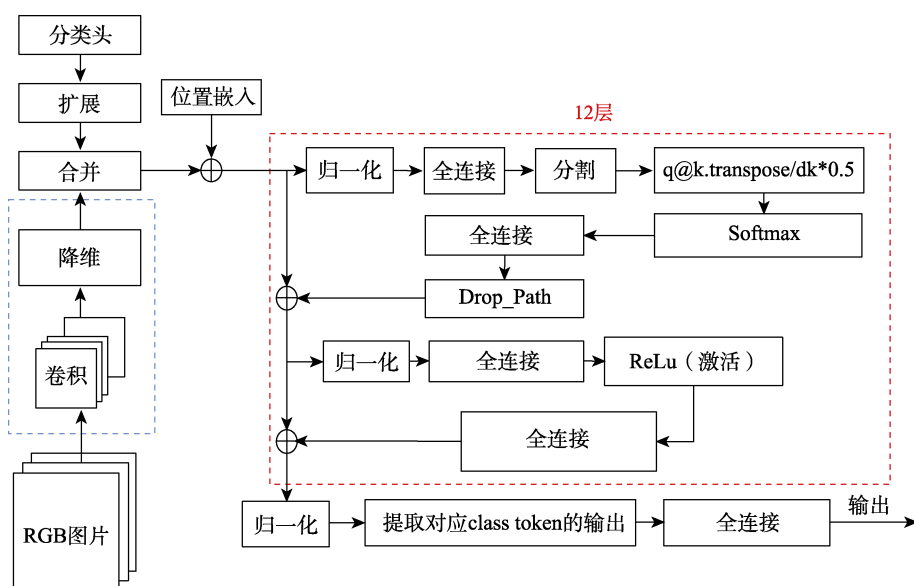


图 4 前向传播程序框架

2.3 参数定点化处理与 Embedding 层评估

针对适合 FPGA 的量化过程, 由于计算过程的模型参数是二进制定点数, 所以模型所有参数结构均需要被量化在合适的二进制位数范围内, 在量化完成后, 再转为二进制, 存于 FPGA 内部的块随机存取存储器 (block random access memory, BRAM)。

量化位数越少, 占用 FPGA 的逻辑资源越少。如果模型的参数为浮点数, 假设原来参数为 b , 定点运算为 1 位符号位、1 位整数, 保留 n 位小数, 先利用 $\text{np.clip}(b, -a, +a)$ 函数将原来参数 b 限定在 $[-a, +a]$ 区间, 即 $c = \text{np.clip}(b, -a, +a)$, 其中 $a = 1 - 0.5^n$, c 为原始参数 b 经 $\text{np.clip}()$ 函数操作的结果, $\text{np.clip}()$ 函数为 numpy 函数库中的钳位函数。

为明确具体定点化 n 的值, 将量化函数应用于模型的训练过程之后、预测过程之前, 将网络参数的各个层量化, 反复修改 n 值的训练过程中, 确定较小的 n 值, 参数定点化程度流程见图 5。

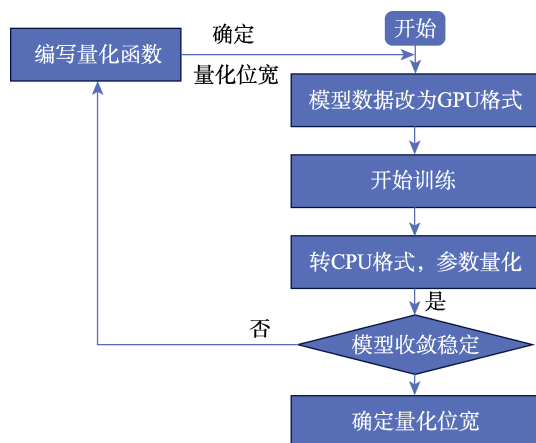


图 5 参数定点化程序流程

Embedding 层的网络参数总计 742 656 个, 参数分析见表 2。完成 Embedding 层评估后, 即可根据模型参数与计算量, 结合系统传输接口设计计算架构。

表 2 Embedding 层参数分析

参数名称	张量维度
cls_token	[1, 1, 768]
pos_embed	[1, 197, 768]
patch_embed.proj.weight	[768, 3, 16, 16]
patch_embed.proj.bias	[768]

3 FPGA 架构设计与芯片选型

3.1 相机接口分析

FPGA 系统采用 OV5640 图像传感器^[20], 配置好传感器驱动后, 图像传感器输出接口时序见图 6: 每帧图像由场同步信号 (vertical synchronization, VSync) 控制, 高电平脉冲代表新的一帧图像开始; 低电平周期传输这一帧图像的像素信息。水平参考信号 (horizontal reference, HREF) 控制整行像素信息的传输, 高电平周期表示时钟脉冲下传输的图像信息有效。本设计的图像帧率为 60 Hz, 像素总量为 1 024 px×768 px。

3.2 预处理单元设计

模型输入图像为 224 px×224 px 的张量, 且像素为 $[-1, 1]$ 区间的值。由于相机采集的图像为 1 024 px×768 px, 故数据需要缩放和归一化后才能用作输入。

在归一化数据时, 可在 8 位无符号的像素值数据的最高位前拼接两位二进制 0, 这两位数字分别表示符号位和整数位。由于像素值为 RGB565 格式, 最低位拼接 0 补成 8 位, 故最后 2 位一定为 0, 加上 -0.5 的带符号二进制补码后, 此时数值在 $(-0.5, 0.5)$ 区间, 最

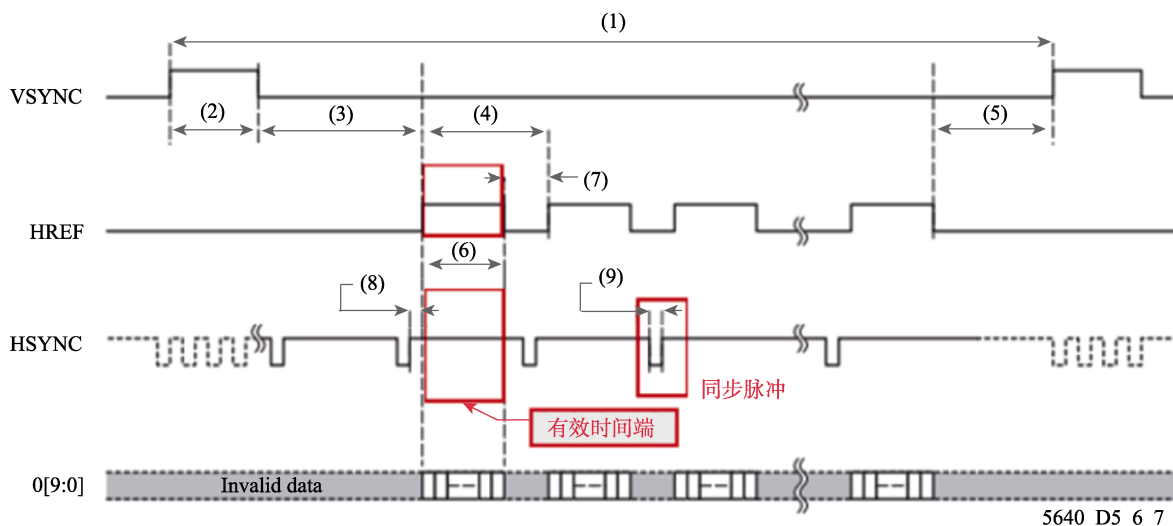


图 6 OV5640 传感器接口时序

高位连续三位的位数值相同, 故可舍弃最高位和最后两位, 以新的数据作为带符号 6 位小数的归一化数组。FPGA 数据归一化操作见图 7。

3.3 向量卷积单元设计

在完成图像预处理操作后, 得到的像素有效时序如图 8 所示, 对原始图像, 从每连续 3 行图像像素中采集 1 行, 在一次有效行内, 像素时钟连续 8 个周期后输出一次有效像素点。

为高效地利用算法模型的输入数据, 可按照卷积核的串行向量, 处理输入的归一化图像数据, 按输入顺序依次将相机串行传输的数据存入双口 RAM 的首地址。

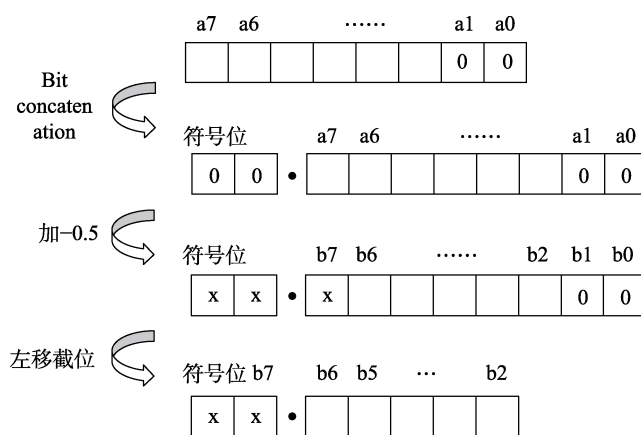


图 7 FPGA 数据的归一化操作

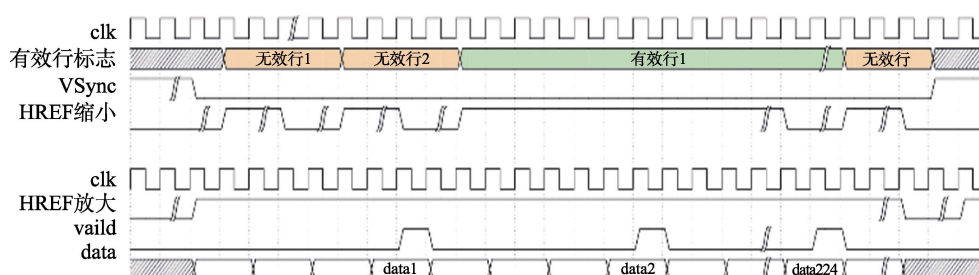


图 8 图像预处理后的像素有效时序

为保证卷积计算的输入数据和权重位置匹配, 在只读存储器 (read only memory, ROM) 单元调整原始模型参数位置顺序后将之存于 ROM 中。在 BRAM 中, 需要按照带符号一位整数的二进制格式将模型参数写入 coe 文件。权重参数对应的 ROM 的参数存储顺序需按照数据输入先后依次递增排列, 其存储内容对应卷积计算过程中输入数据所需的权重值, 该处理单元的输入输出行为可抽象为如图 9 所示的卷积单元

IO 位序图。若下一时钟周期所需的卷积核窗口位置尚未被覆盖, 则其对应权重将被拼接并存入新的地址。此过程持续进行, 直至整个卷积核窗口的所有位置均被覆盖。

在为整个处理架构建模时, 需要对资源和时间进行计算约束。在确定卷积核宽度为 16、存满 2 个卷积核宽度大小的数据时, 需要确定数据缓存完成的时间间隔。设像素时钟周期为 48 MHz, 则根据图 8 的有效信

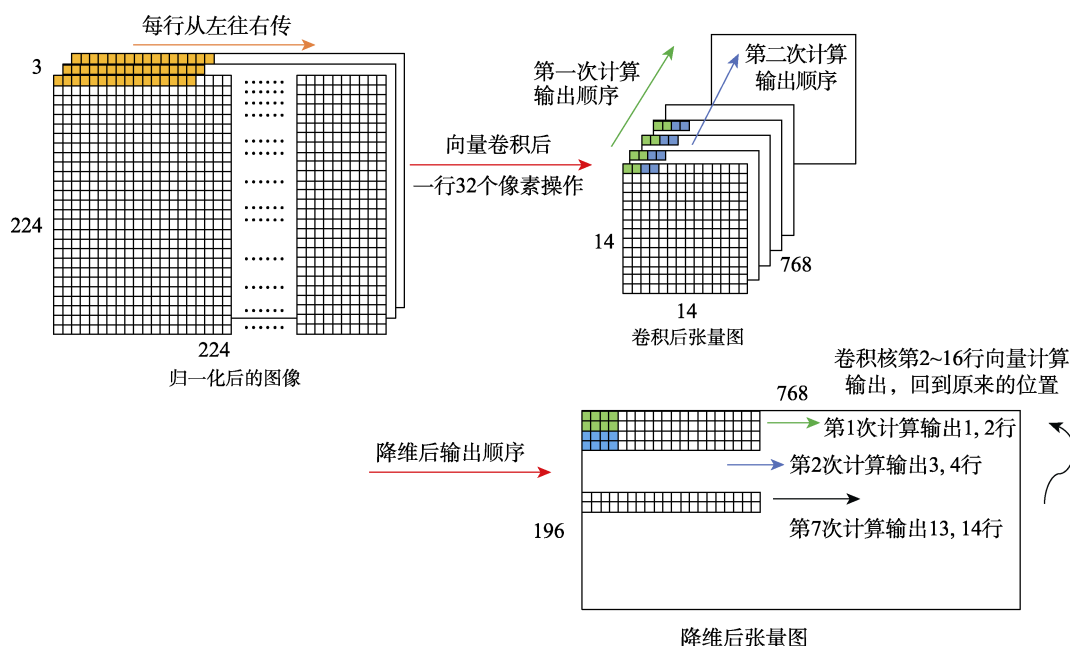


图 9 卷积单元 IO 位序图

号时序图可知,数据缓存完成的时间间隔 $T_{\text{缓存}}$ 如下。

$$T_{\text{缓存}} = \frac{1}{48} \times 32 \times 4 \times 2 \quad (1)$$

为确定卷积计算的时钟周期,同时保证 FPGA 资源的合理开销,需要假设输入向量一个周期的计算深度为 h 、计算单元的计算频率为 f (单位: MHz),由于该层网络深度为 768,故一次计算任务完成时间 $T_{\text{计算}}$ 如下。

$$T_{\text{计算}} = \frac{1}{f} \cdot \frac{768}{h} \quad (2)$$

同时可得计算单元的 BRAM 字节用量 $M_{\text{卷积}}$ 估计值如下。

$$M_{\text{卷积}} = L \cdot h \cdot H \cdot 2 + M_{\text{权重总数}} \quad (3)$$

式中: $M_{\text{权重总数}}$ 为卷积权重的总参数量; L 为卷积向量的宽度,在模型中值为 16; H 为输入图像通道数,值为 3。

图 10 展示了向量卷积处理模型,包括向量缓存 RAM 单元、计算控制单元、卷积权重存储 ROM 单元、向量卷积阵列、寄存器组等。计算控制单元的作用是归一化后的传感器用户接口的行场同步信号和有效信号,实现对数据 RAM 和参数 ROM 的地址输出与使能控制(地址的输出和有效使能均为对有效信号的计数),同时实现控制信号的跨时钟域处理。

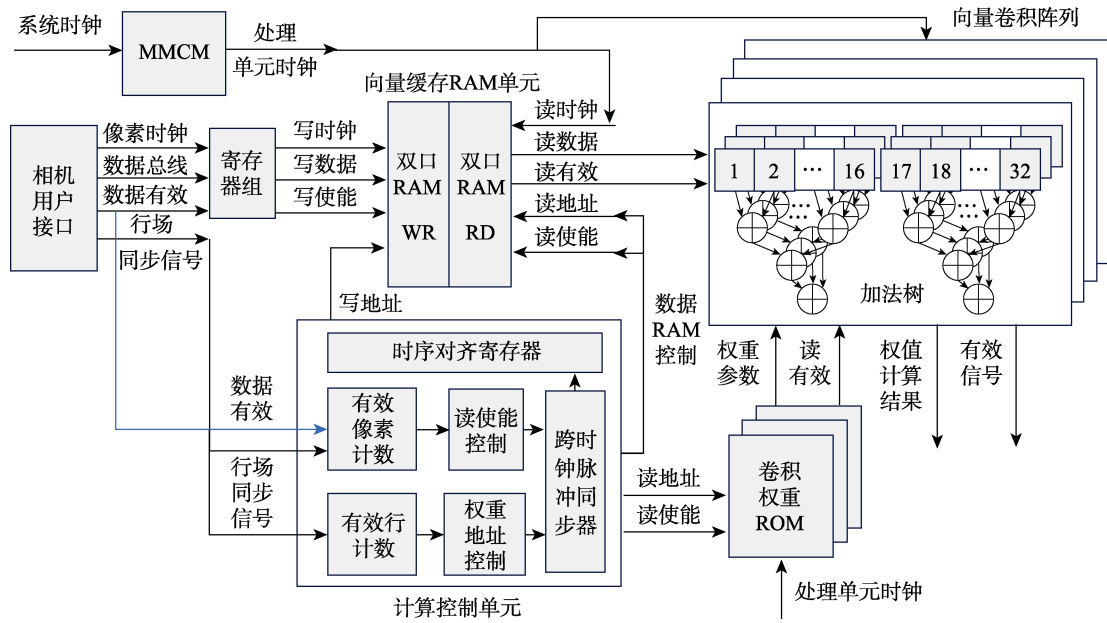


图 10 向量卷积处理模型

3.4 累计单元设计

由于向量输出结果并不完整,在完成卷积操作后,需要通过累积叠加将处理后的结果补充完整。剩余的模型参数如表 3 所示。在得到图 9 中的降维输出张量后,需要叠加和拼接表 3 中的剩余参数,再进行后续的网络计算。为了节省 BRAM 的存储资源,以及减少数据运算量,在叠加前需要事先整合参数。首先,从复制二维的方式扩展偏置参数,将 `cls_token` 拼接于 `pos_embed` 的首位,然后,与二维扩展后的偏置进行叠加操作,最后将整个张量分为头张量和体张量,过程见图 11。

表 3 剩余参数

参数名称	张量维度
<code>cls_token</code>	[1, 1, 768]
<code>pos_embed</code>	[1, 197, 768]
<code>patch_embed.proj.weight</code>	[768]

按照图 9 对应的输出顺序为图 11 的体张量配置 `coe` 文件参数的位宽和地址深度,在第 1 行卷积核向量得到输出结果叠加的配置参数,在第 2 行到 16 行的卷积向量核叠加上一层的输出和之前同地址下累计寄存的结果,向量均叠加计算完成后,按照单行的输出顺序输出存储的值,将该值传输到后续的网络传输接口。根据上述动作,累计单元架构模型如图 12 所示。

在该模型中,由于写入数据带宽为读出数据带宽的两倍,在数据叠加完整后,再输出数据。若忽略动作响应周期,则完整输出串行总时间 $T_{\text{累出}}$ 将为

$$T_{\text{累出}} = \frac{768}{h} \cdot 14 \cdot \frac{1}{f} \quad (4)$$

在完成当前累计输出后,启动下一次卷积计算需要先接收两行新输入数据,最少等待时间 $T_{\text{传输}}$ 为

$$T_{\text{传输}} = 3 \times 1024 \times 2 \times \frac{1}{48} \quad (5)$$

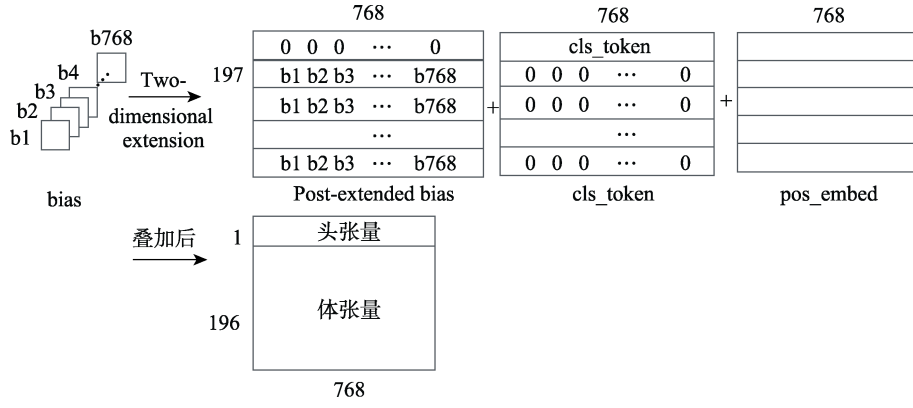
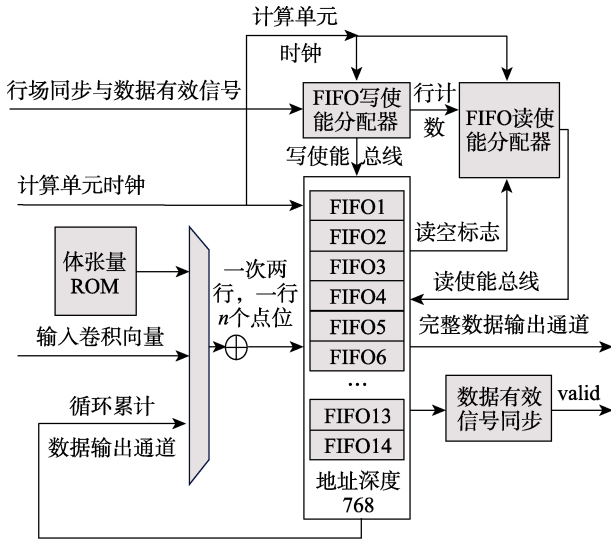


图 11 偏置参数处理过程



注: FIFO 为 first-in-first-out, 即先进先出, 一种队列处理规则。

图 12 累计单元架构模型

相关的存储资源用量 $M_{\text{累出}}$ 为

$$M_{\text{累出}} = M_{\text{体}} \quad (6)$$

式中: $M_{\text{体}}$ 为图 11 中体张量所占字节数。

3.5 传输缓存单元设计

本设计采用了 UDP 传输方案^[20], 封装的数据将通过 UDP 层传输到 IP 层, 在 MAC 层被再次组包后, 通过千兆以太网传输。本设计的 UDP 协议栈用户接口为单字节传输, 计算输出中整数和小数各占 1 B (bytes)。由于 UDP 数据包一次最多传输 1 472 B, 由图 9 中降维输出张量可得, 一行向量计算数据为 768 B, 故一次传输无法输出一行完整的计算数据。

本设计为保证 UDP 传输单个数据的完整性, 以及分片传输长度的一致性, 且为确保后续处理单元能获得向量在张量中的位置, 需要统计场同步信号和输入有效信号中的高电平脉冲次数, 得到数据所在张量的行号。将一行数据分两次传输, 并以字节标记数据属于前半行还是后半行, 依次拼接标记字节和行号字节, 将之嵌入累计输出数据有效段之前, 最后将拼接后的数据作为 UDP 报文传输。由于 UDP 用户时钟和卷积处理单元时钟频率、读写时钟和读写位宽均不同, 在 UDP 协议栈输入前需要以 FIFO 的方式连接存储单元, 传输缓存单元见图 13。

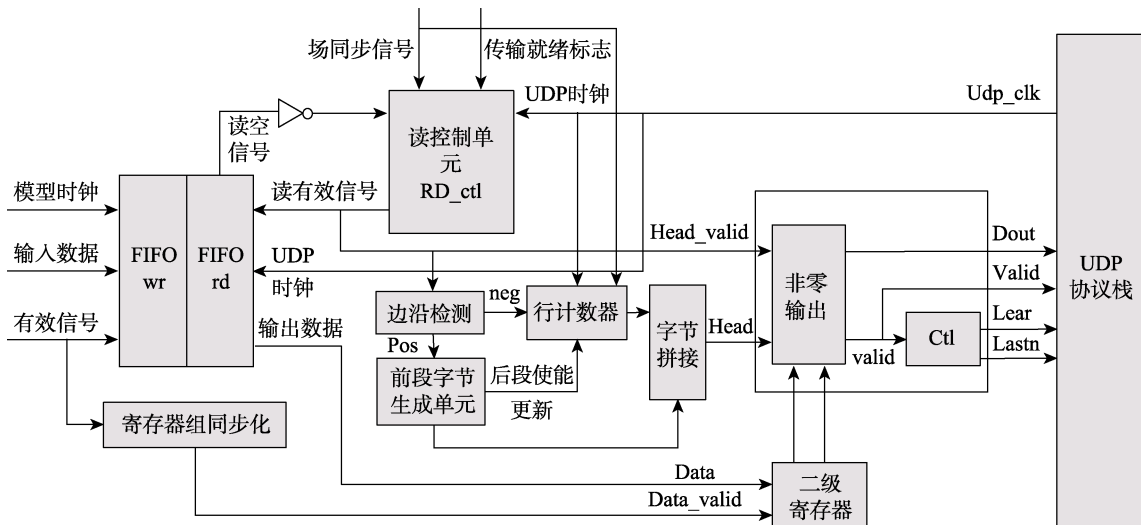


图 13 传输缓存单元

在该模型中, FIFO 读出时间慢于写入时间, 在写入不为空时, 通过 FIFO 缓存输出数据, 由于协议栈时钟为 125 MHz, FIFO 数据输出为单字节串行输出, 则数据发送间隔 $T_{\text{发送}}$ 的值有

$$T_{\text{发送}} = 768 \times 2 \times 14 \times \frac{1}{125} \quad (7)$$

记下一次传送需要的时间周期为 $T_{\text{udp_ready}}$, 其值有

$$T_{\text{udp_ready}} = 16 \times 1024 \times 2 \times \frac{1}{48} \quad (8)$$

相关的存储资源用量 $M_{\text{发送}}$ 的值有

$$M_{\text{发送}} = M_{\text{体}} \quad (9)$$

3.6 资源时钟评估与芯片选型

根据系统划分和建模过程, 整合式(1)一式(9), 则有

$$\begin{cases} T_{\text{缓存}} > T_{\text{计算}} \\ T_{\text{累出}} < T_{\text{传输}} \\ T_{\text{发送}} < T_{\text{udp_ready}} \\ M_{\text{总}} = M_{\text{卷积}} + M_{\text{累出}} + M_{\text{发送}} \end{cases} \quad (10)$$

解上述约束得

$$\begin{cases} h \cdot f > 144 \\ M_{\text{总}} = 96 \cdot h + 497\,664 \end{cases} \quad (11)$$

上述计算处理单元建模对并行量 n 和计算时钟频率有解, 且对 FPGA 系统而言, h 的最小值为 1, 时钟频率 f 至少要求 144 MHz, 而 144 MHz 远未超过 FPGA 工艺的时钟频率物理上限。本设计采用 250 MHz 的时钟频率处理计算单元, n 取 4。

为保证执行其他任务的逻辑资源量, 本设计采用 ZYNQ7045 芯片管理资源。在设计完成后, 芯片内部各个资源利用情况见图 14。

Resource	Utilization	Available	Utilization %
LUT	46433	218600	21.24
LUTRAM	1010	70400	1.43
FF	45535	437200	10.42
BRAM	307.50	545	56.42
DSP	2	900	0.22
IO	113	362	31.22
BUFG	16	32	50.00
MMCM	5	8	62.50
PLL	1	8	12.50

图 14 芯片内部各个资源利用情况

Design Timing Summary			
Setup	Hold	Pulse Width	
Worst Negative Slack (WNS): -0.513 ns	Worst Hold Slack (WHS): 0.044 ns	Worst Pulse Width Slack (WPWS): 1.100 ns	
Total Negative Slack (TNS): -501.096 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns	
Number of Failing Endpoints: 3438	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	
Total Number of Endpoints: 39368	Total Number of Endpoints: 39269	Total Number of Endpoints: 14822	
Timing constraints are not met.			

图 16 布局布线时出现的时序违例报告

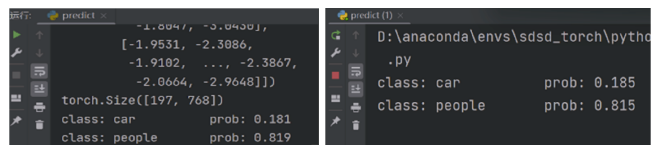
4 仿真验证与顶层优化

4.1 仿真验证

首先, 利用 Python 程序将 1 024 px × 768 px 的图像数据分为三通道 (RGB565), 分别以二进制的形式存入文本文件, 每个文件存储一个通道, 均含 768 行数据, 每行数据中, 每一个像素分别占 (5, 6, 5) bit, 每一行共存 1 024 px 数据, 且像素与像素之间直接拼接, 无分隔符。

然后, 构建 testbench 模型, 根据 OV5640 技术手册的接口时序模拟接口时序输入, 为验证模型计算结果, 初始化存储模型的文本文件, 在输出结果有效时, 按照次序存储输出的数据, 在输出完成一帧图像的计算结果时停止仿真, 将得到的结果输入 Python 程序中, 读取文本文件, 将数据转化为浮点 torch 格式后, 在数据开头添加头张量, 进行后续的计算。

图 15 为基于 pytorch 的编译验证结果和 Verilog 编译验证结果的对比, 由于原始模型未被转换为 1 024 px × 768 px, 数据的计算结果存在一定的误差。



(a) 基于pytorch的编译验证结果

(b) Verilog编译验证结果

图 15 验证结果的对比

4.2 时序与资源优化

布局布线时出现的时序违例报告见图 16, 说明数据线路的建立时间不满足要求。

根据 EDA 报告分析结果, 第一类设计因乘法器链路逻辑级数过高而存在时序问题 (图 17)。为此, 可将流水线深度从 1 级增至 3 级以解决该问题。

第二类设计问题是权重参数使用的 BRAM 较为集中, 导致布线拥挤及时序违例。为解决该问题, 需要为 ROM 最小单元的划分配置方案选择方案尺寸与卷积向量尺寸的最小公倍数最接近的配置, 图 18 展示了布线拥挤来源。

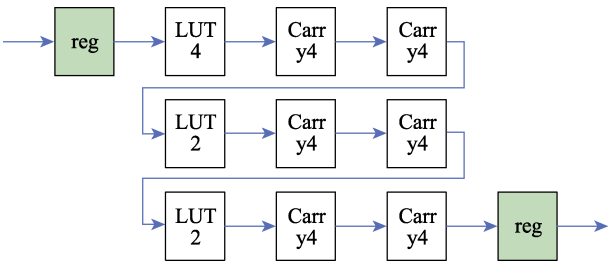


图 17 乘法链路问题

修正上述两类问题后，布局布线结果见图 19，在建立时间中，最差负延迟时间为 0.128 ns；在保持时间中，最差等待时间为 0.048 ns，均大于零，故本设计可满足时序要求。

图 20 为优化前后的布局布线图，由图中资源布局可知，整体资源自左向右，自上向下，符合 IO 接口顺序布局要求。

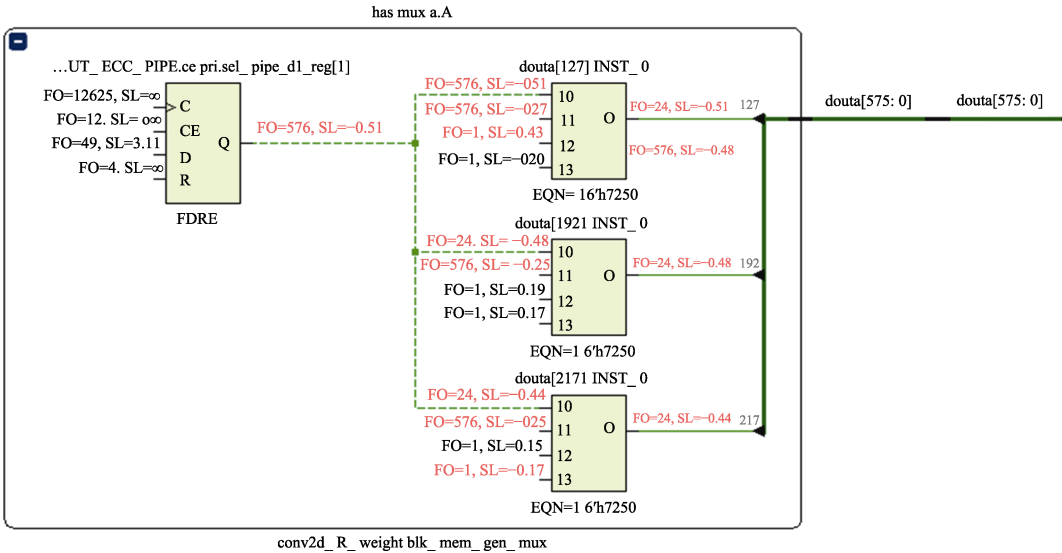
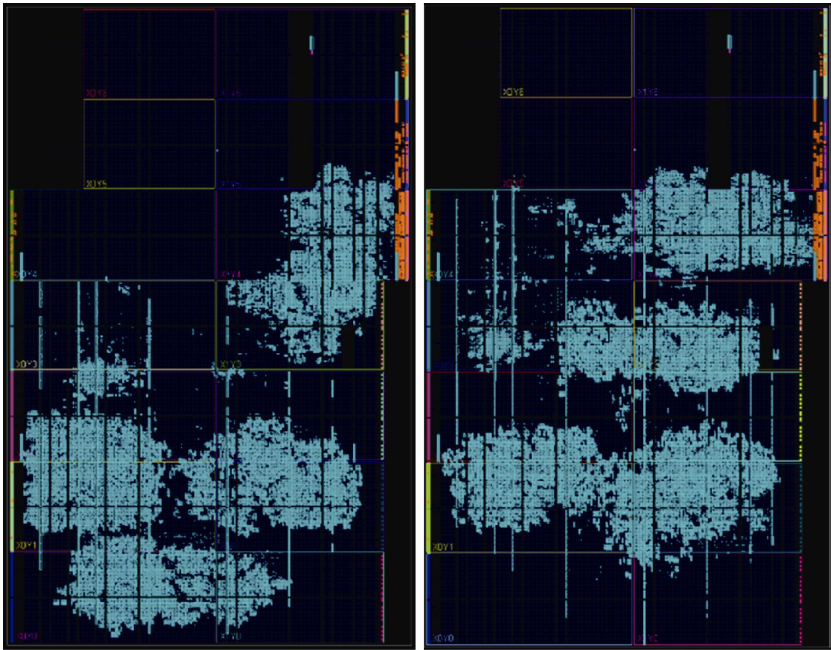


图 18 布线拥挤来源

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 0.128 ns	Worst Hold Slack (WHS): 0.048 ns	Worst Pulse Width Slack (WPWS): 1.100 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 83610	Total Number of Endpoints: 83511	Total Number of Endpoints: 38692
All user specified timing constraints are met.		

图 19 优化后的时序报告



(a) 优化前的布局布线图 (b) 优化后的布局布线图

图 20 优化前后的布局布线图

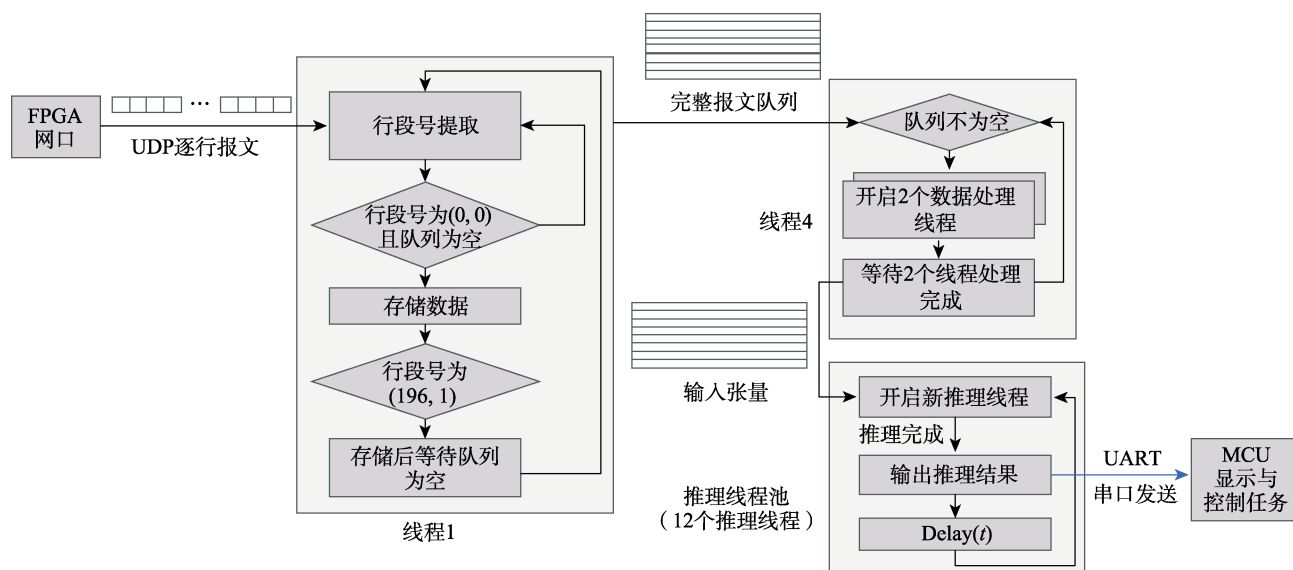
5 多线程设计与运行结果

为方便利用 FPGA 处理和传输的计算结果, 本工作先在 PC 端实地运行, 再将程序迁移至 NPU 进行开发。首先, 单独提出原始模型的深层结构, 将所有使用 torch 库函数的程序归于模型文件, 构建 deep_model.py 程序文件, 然后将图 11 所述的头张量存于后续输入存储的列表开头, 最后, 初始化 UDP 协议栈, 绑定本地 IP 和端口号, 开始接收数据。

创建第一个线程, 由该线程接收 FPGA 发送的 UDP 报文, 将报文存入队列, 启动线程使能 UDP: 将接收的十六进制数据转为二进制, 提取第一个和第二个字节数据, 将之转化为前后端标志和行号, 在队列为空、数据行号和首字节都为 0 时, 将数据存于队列中, 在数据行号和首字节都为预设的最大值时停止存入队列, 这个操作确保了即便是在写快读慢的情况下, 队列存储也不会出现累计延迟, 并确保了数据存帧的完整性。

启动线程 1 后, 程序进入主循环。在每次循环迭代中, 主线程首先从完成队列中读取一个数据段。然后, 创建并启动线程 2 和线程 3: 线程 2 负责处理该数据段前半段的二进制数据 (包含整数和小数部分), 将其转换为十进制格式并存储到分配给它的前半段 (整行数据中第 1~512 个数据字节) 四分之一位置; 线程 3 则负责处理该数据段后半段的二进制数据 (包含整数和小数部分), 将其转换为十进制格式并存储到分配给它的后半段 (整行数据中第 513~1 024 个数据字节) 四分之一位置。待线程 2 和线程 3 完成各自任务后, 主线程读取当前数据段头部的第 1 个和第 2 个字节 (包含控制信息), 并基于此信息启动下一轮数据的转化和存储操作。同时, 主线程检查当前处理的数据段是否是该行最后 1 个片段 (即行号最大且段标最后)。若满足此条件, 则启动模型的深层网络计算任务, 调用 NPU 进行计算。图 21 为线程处理程序流程。

经过多线程和线程池处理, 识别结果由图像形式转换为 NPU 运行的结果, 见图 22, 对人和车的识别精



注: Delay(t) 表示延迟时间, t 为单线程配置下算法推理所用时间除以多线程配置下线程总数所得结果。

图 21 线程处理程序流程

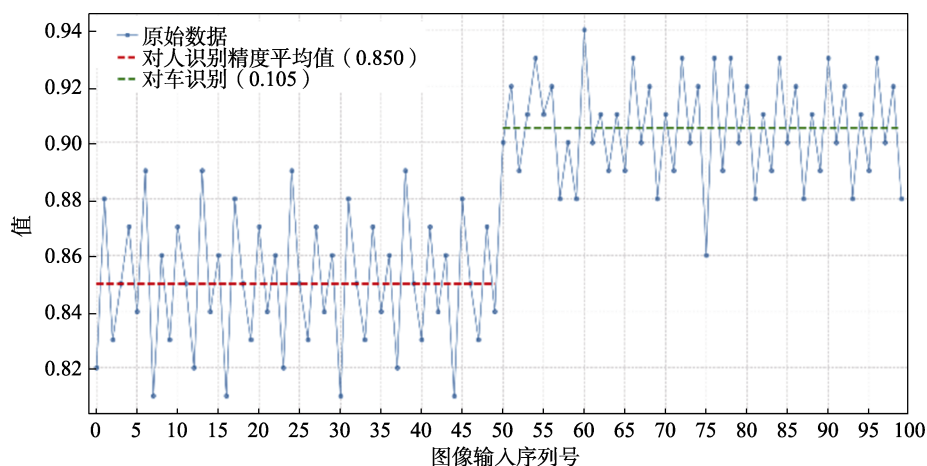


图 22 分类效果

度平均值分别为 85%、90.5%。由图 23 可知, 模型每帧 平均运行时间由最初的 0.643 9 s 提升至 0.099 5 s。

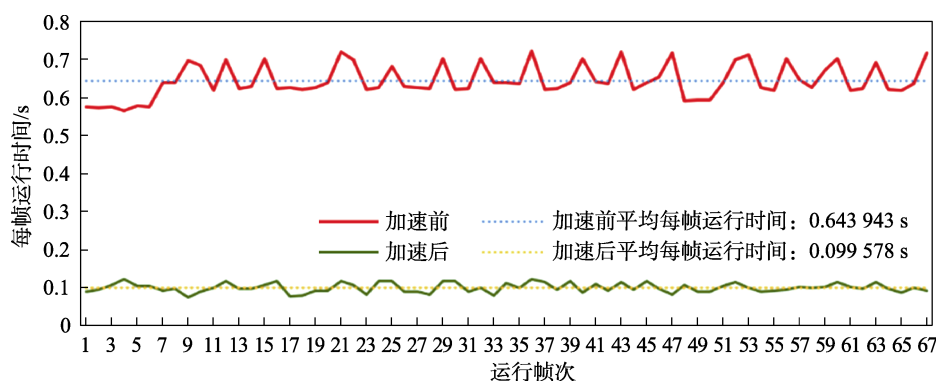


图 23 每帧数据运行时间

参考文献 (References)

- [1] DOSOVITSKIY A, BEYER L, KOLESNIKOV A, et al. An image is worth 16x16 words: Transformers for image recognition at scale[Z/OL]. (2020-10-22) [2025-04-08]. <https://arxiv.org/pdf/2010.11929>.
- [2] HAN K, WANG Y H, CHEN H T, et al. A survey on vision transformer[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2022, 45(1): 87–110.
- [3] VASWANI A, SHAZEER N, PARMAR N, et al. Attention is all you need[J]. Advances in Neural Information Processing Systems, 2017, 30.
- [4] LIU Z, LIN Y, CAO Y, et al. Swin transformer: Hierarchical vision transformer using shifted windows[C]//Proceedings of the IEEE/CVF International Conference on Computer Vision. New York, NY: Institute of Electrical and Electronics Engineers, 2021: 10012–10022.
- [5] TOUVRON H, CORD M, DOUZE M, et al. Training data-efficient image transformers & distillation through attention[C]//International Conference on Machine Learning. New York: Proceedings of Machine Learning Research, 2021: 10347–10357.
- [6] KHAN S, NASEER M, HAYAT M, et al. Transformers in vision: A survey[J]. ACM Computing Surveys (CSUR), 2022, 54(10s): 1–41.
- [7] SZE V, CHEN Y H, YANG T J, et al. Efficient processing of deep neural networks: A tutorial and survey[J]. Proceedings of the IEEE, 2017, 105(12): 2295–2329.
- [8] NURVITADHI E, VENKATESH G, SIM J, et al. Can FPGAs beat GPUs in accelerating next-generation deep neural networks?[C]//Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays. New York: Association for Computing Machinery, 2017: 5–14.
- [9] ZHANG C, LI P, SUN G Y, et al. Optimizing FPGA-based accelerator design for deep convolutional neural networks[C]//Proceedings of the 2015 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays. New York: Association for Computing Machinery, 2015: 161–170.
- [10] ZHAO R, NIU X, WU Y, et al. Optimizing CNN-based object detection algorithms on embedded FPGA platforms[C]//2017 IEEE International Symposium on Circuits and Systems (ISCAS). New York: Institute of Electrical and Electronics Engineers, 2017: 1–4.
- [11] HOWARD A G, ZHU M, CHEN B, et al. MobileNets: Efficient convolutional neural networks for mobile vision applications [Z/OL]. (2017-04-17) [2025-04-08]. <https://arxiv.org/pdf/1704.04861>.
- [12] SHEN J, HUANG Y, WANG Z, et al. Towards a uniform template-based architecture for accelerating 2D and 3D CNNs on FPGA[C]//Proceedings of the 2018 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays. New York: Association for Computing Machinery, 2018: 97–106.
- [13] CHEN Y H, KRISHNA T, EMER J S, et al. Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks[J]. IEEE Journal of Solid-State Circuits, 2016, 52(1): 127–138.
- [14] ALWANI M, CHEN H, FERDMAN M, et al. Fused-layer CNN accelerators[C]//Proceedings of the 49th Annual IEEE/ACM International Symposium on Microarchitecture. New York: Institute of Electrical and Electronics Engineers, 2016: 1–12.
- [15] MA Y F, CAO Y, VRUDHULA S, et al. Optimizing loop operation and dataflow in FPGA acceleration of deep convolutional neural networks[C]//Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays. New York: Association for Computing Machinery, 2017: 45–54.
- [16] DING W, HUANG Z Y, HUANG Z K, et al. Designing efficient accelerator of depthwise separable convolutional neural network on FPGA[J]. Journal of Systems Architecture, 2019, 97: 278–286.
- [17] CHEN Y H, KRISHNA T, EMER J S, et al. Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks[J]. IEEE Journal of Solid-State Circuits, 2016, 52(1): 127–138.
- [18] LI E, ZENG L, ZHOU X, et al. Edge AI: On-demand accelerating deep neural network inference via edge computing[J]. IEEE Transactions on Wireless Communications, 2020, 19(1): 447–457.
- [19] POSTEL J. User datagram protocol: RFC 768[R]. Los Angeles, CA: Institute for Scientific Information, 1980.
- [20] OmniVision Technologies. OV5640 1/4-inch CMOS UXGA camera chip sensor datasheet[Z]. Rev 1.3-ed. Santa Clara: OmniVision Technologies, 2011.

Image classification processing system based on FPGA and RK3588

YUAN Xiaoping, TU Qiyang

(School of Information and Control Engineering, China University of Mining and Technology, Xuzhou 221116, China)

Abstract: [Objective] Vision transformer (ViT) models have achieved superior performance in image classification tasks owing to their global attention mechanisms. However, their high computational complexity and resource-intensiveness hinder their deployment on resource-constrained edge devices such as autonomous systems or robotics that demand low latency and high efficiency. Traditional CPU-based platforms pose issues such as prolonged processing delays, inefficient resource utilization, and limited parallel processing capabilities, struggling to meet the real-time requirements for ViT inference. These limitations are exacerbated in scenarios requiring high frame rates (e.g., 60 Hz) and low power consumption. Therefore, a hybrid field-programmable gate array (FPGA) and neural processing unit (NPU) architecture was designed to accelerate ViT-based image classification while maintaining high accuracy and low latency. This system bridges the gap between the theoretical advantages of ViT and practical deployment constraints in edge computing environments. [Methods] The system architecture integrated the FPGA and RK3588 NPU to optimize its computational efficiency via hardware codesign. The FPGA handled the image acquisition, preprocessing, and initial layers of the ViT model (e.g., patch embedding and positional encoding). Key optimizations included the following: ①Vectorized Computation: The patch embedding layer was decomposed into vectorized operations, leveraging the parallel processing capabilities of the FPGA. ②Pipeline Design: A pipelined architecture merged tensor operations for cumulative output, reducing intermediate data storage and latency. ③UDP-Based Data Transmission: The FPGA-processed results were transmitted to the RK3588 NPU via the UDP protocol, ensuring real-time data flow and synchronizing tasks such as image display and classification. ④NPU Acceleration: The RK3588 NPU employed multithreading and thread-pooling techniques for attention computation and deep-layer processing. Second, the design workflow involved algorithm partitioning between the FPGA and RK3588 NPU, fixed-point quantization for FPGA compatibility, and rigorous simulation to validate time and resource constraints. For the FPGA implementation, model parameters were quantized to an eight-bit fixed-point format to reduce BRAM usage by 34% and maintain classification accuracy within 1.5% of the floating-point baseline. Hardware optimizations such as BRAM resource allocation and timing adjustments were implemented to eliminate setup violations and improve throughput. Postlayout simulations confirmed that the worst-case setup and hold times met the FPGA time margins (0.128 and 0.048 ns). [Results] The experimental validation demonstrated considerable improvements across three key metrics: ①Latency Reduction: The average per-frame processing time decreased from 0.643 9 s (CPU baseline) to 0.099 5 s, achieving real-time performance at 60 Hz, representing an 84.5% reduction in latency. This enabled seamless integration into time-sensitive applications such as autonomous navigation. ②Resource Efficiency: FPGA BRAM utilization reached 56.42% with no timing violations after optimization. The design achieved a logic cell utilization of 72.3% and DSP slice utilization of 64.8%, demonstrating efficient resource allocation. The power consumption was measured at 3.2 W, considerably lower than that of GPU-based solutions (typically >20 W). ③Classification Accuracy: The system achieved average precision scores of 0.85 (human) and 0.905 (vehicle) on a custom dataset containing 10 000 annotated images. These results validated the effectiveness of hardware–software codesign, with a 1.2% accuracy degradation compared with the original floating-point model. [Conclusions] By combining the parallel processing strengths of FPGA with high-throughput computing of NPU, this study successfully addressed the inefficiencies of ViT deployment on edge platforms. This architecture enhanced real time performance and maintained high accuracy, making it suitable for applications such as autonomous driving, industrial automation, and smart surveillance. The key innovations included the implementation of vectorized patch embedding, hybrid UDP–NPU task scheduling, and optimized fixed-point quantization. Future work will focus on expanding the model scalability to support large ViT variants (e.g., ViT-Large), exploring dynamic resource-sharing mechanisms between the FPGA and NPU, and integrating lightweight compression techniques to further reduce the memory footprint.

Key words: FPGA; RK3588; vectorized computation; superposition fusion; location identification message; multi-threaded design

(编辑: 孙浩)